

پروژه پایانی درس سیستم های عامل

فرزان مسیپی

400522175

مبین آزادانی

400521027

بهمن 1402

پیاده سازی:

پیاده سازی تابع clone():

[Type here]

:int clone(int (*func)(void *args), void *child_stack, int flags, void *args)

برای ایجاد thread های جدید به کار میرود. این تابع با wrap کردن تابع اصلی copy_process یک رشته ایجاد می‌کند و در صورت موفق بودن tid آن را برمی‌گرداند.

در ابتدا با صدا زدن تابع allocproc() یک struct proc جدید ایجاد خواهد شد. در صورت عدم موفقیت در ایجاد تابع مقدار ۱- برمی‌گرداند. سپس در صورتی که آدرس استک مربوط به thread جدید به تابع داده شده باشد، از روی آن و در غیر اینصورت، برنامه به صورت خودکار به thread جدید حافظه‌ای به عنوان استک اختصاص می‌دهد. پس از آن، context مربوط به thread فراخوانده تابع، به thread جدید کپی میشود. اطلاعات کپی شده شامل محتوای ثبات ها، stack pointer، اطلاعات

```
262 // child stack for execution
263
264 // child process sharing the virtual address space
265 if((CLONE_VM & flags)){
266     // the child process needs stack for execution
267     if(!child_stack){
268         // guard page of the group leader thread
269         guard_page = tleader->tstack - 2 * PGSIZE;
270         // modify the page directory entry by extending the virtual address
271         if((np->tstack = clonevm(tleader->pgdir, tleader->sz, guard_page)) == 0){
272             kfree(np->kstack);
273             np->kstack = 0;
274             np->state = UNUSED;
275             return -1;
276         }
277         // the stack is allocated by the kernel
278         np->tstackalloc = 1;
279     }
280     // allocated stack for child process
281     else{
282         np->tstack = (char *)child_stack;
283     }
284 }
285 // child process not sharing the virtual address space
286 else{
287     np->tstack = tleader->tstack;
288     // thread trying to clone without sharing any virtual address space
289     if(curproc != tleader){
290         if(copy_thread_stack(np->pgdir, np->tstack - PGSIZE,
291             tleader->pgdir, curproc->tstack - PGSIZE) == -1) {
292             return -1;
293         }
294     }
295 }
```

[Type here]

مربوط به ofile و ... میباشد. در نهایت حالت thread ایجاد شده به حالت RUNNABLE تغییر پیدا میکند.

```
330 // change of execution point depending upon the flag passed
331 np->tf->eip = (uint)func; // change instruction pointer for execution
332 np->tf->esp = sp; // change stack pointer for execution
333
334 for(uint i = 0; i < NOFILE; i++){
335     if(curproc->ofile[i]){
336         // duplicate all the file descriptors
337         if((flags & CLONE_FILES)){
338             np->ofile[i] = filedup(curproc->ofile[i]);
339         } else{
340             np->ofile[i] = filealloc();
341             np->ofile[i]->type = curproc->ofile[i]->type;
342             np->ofile[i]->ip = idup(curproc->ofile[i]->ip);
343             np->ofile[i]->off = 0;
344             np->ofile[i]->readable = curproc->ofile[i]->readable;
345             np->ofile[i]->writable = curproc->ofile[i]->writable;
346         }
347     }
348 }
349
350 if((flags & CLONE_FS)){
351     np->cwd = idup(curproc->cwd);
352 } else{
353 }
354
355 // name of the child process is same as that of the original process
356 safestrcpy(np->name, curproc->name, sizeof(curproc->name));
357
358 if((flags & CLONE_THREAD)){
359     // child process formed would be thread, thus pid is same as thread id
360     np->tid = np->pid;
361
362     // pid of the clone child process is same as the parent process
363     np->pid = curproc->pid;
364
365     // return id is the thread id
366     retid = np->tid;
367 } else{
368
369     // the child process is thread leader in different thread group
370     np->tid = -1;
371
372     // return id is the process id
373     retid = np->pid;
374 }
```

```
376 acquire(&table.lock);
377
378 // child process state is set to runnable for scheduling
379 np->state = RUNNABLE;
380
381 release(&table.lock);
382
383 // returns the thread id of the child process
```

[Type here]

موارد مربوط به کپی کردن اطلاعات، بر اساس flag هایی انجام میشوند که به تابع به عنوان ورودی داده میشود (int flags).

هر flag برای عمل خاصی اختصاص داده شده است که به ترتیب زیر هستند:

flag CLONE_VM: اگر مقدار این flag ۱ شود، نشان دهنده این است که هر دو thread از حافظه اشتراکی استفاده میکنند. در غیر اینصورت حافظه دو thread جدا از هم خواهد بود.

flag CLONE_FS: اگر مقدار این flag ۱ شود، file system مربوط به thread جدید و thread والد یکسان خواهد بود.

flag CLONE_FILES: اگر مقدار این flag ۱ شود، file descriptor مربوط به thread جدید و thread والد یکسان خواهد بود.

flag CLONE_THREAD: اگر مقدار این flag ۱ شود، thread جدید یک عضو از گروه thread والد خواهد بود.

```
1 // CLONE_VM : flag is set when the child process and parent process share same
2 //     memory region, else child executes in different virtual address space
3 #define CLONE_VM      (1)
4
5 // CLONE_FS : flag is set, when the child process clones the file system
6 //     information from parent process
7 #define CLONE_FS      (2)
8
9 // CLONE_FILES : flag is set when child process clones the file descriptors
10 //     information from parent process
11 #define CLONE_FILES   (4)
12
13 // CLONE_THREAD : the child process becomes a thread in the parent process
14 //     thread group
15 #define CLONE_THREAD  (8)
16
17 #define CLONE_PARENT  (16)
18
19 // macro that can be used to create thread by passing
20 // appropriate flags arguments to clone system call
21 #define TFLAGS        (CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_THREAD)
22
```

پیاده سازی تابع join():

این تابع برای منتظر ماندن یک thread برای اتمام thread دیگر استفاده می شود.

[Type here]

به عنوان ورودی یک tid می گیرد که همان آیدی thread است که thread فراخواننده تابع منتظر اجرای آن است.

به اینصورت که اگر thread متناظر با tid در آرایه ptable موجود باشد، مقدار thread_join_exits برابر با یک خواهد شد. پس از آن مقدار thread_join_exit و curproc-killed بررسی می شود. اگر هرکدام از آنها یک باشد (یا thread یافت نشده یا متعلق به گروه متفاوتی است) تابع مقدار ۱- را برمیگرداند.

در غیر اینصورت تابع lock را به دست گرفته و مقدار killed مربوط به پراسس فعلی را بررسی کرده و پس از آن شرط zombie بودن thread متناظر با tid را بررسی کرده و در نهایت عملیات آزاد کردن حافظه مربوط به thread متناظر با tid را آغاز کرده و حالت آن را به UNUSED تغییر میدهد.

```
505 join_thread_exits = 0;
506 // check if the thread joining the tid both belong to same thread group
507 for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
508     if(p->tid == tid && p->parent == tleader) {
509         join_thread_exits = 1;
510         break;
511     }
512 }
513
514 // join thread either doesn't exists or it doesn't belong to same group
515 if(!join_thread_exits || curproc->killed){
516     return -1;
517 }
```

```
521 // suspend execution of current thread and wait for completion of tid thread
522 for(;;){
523     // thread is killed by some other thread in group
524     if(curproc->killed){
525         release(&ptable.lock);
526         return -1;
527     }
528     if(p->state == ZOMBIE){
529         // Found the thread
530         jt看id = p->tid;
531         kfree(p->kstack);
532         p->kstack = 0;
533         if(p->tstackalloc){
534             freeclonevm(p->pgdir, p->tstack);
535         }
536         p->pgdir = 0;
537         p->pid = 0;
538         p->tid = 0;
539         p->tstack = 0;
540         p->parent = 0;
541         p->name[0] = 0;
542         p->killed = 0;
543         p->state = UNUSED;
544         release(&ptable.lock);
545         return jt看id;
546     }
547     // Wait for thread to complete (See wakeup1 call in proc_exit.)
548     sleep(tleader, &ptable.lock);
549 }
550 return -1;
```

پیاده سازی تابع thread_create():

این تابع برای ایجاد thread جدید استفاده میشود. در صورت موفقیت عدد صفر و در غیر اینصورت عدد 1- را برمیگرداند.

[Type here]

```
68 int
69 thread_create(kernel_thread_t *kernel_thread, int func(void *args), void *args)
70 {
71     kernel_thread->state = NEW;
72
73     // allocate child stack
74     if(create_thread_stack(&(kernel_thread->tstack)) == -1) {
75         return -1;
76     }
77
78     // cannot create thread
79     if((kernel_thread->tid = clone(func, kernel_thread->tstack, TFLAGS, args)) == -1) {
80         destroy_thread_stack(&(kernel_thread->tstack));
81         kernel_thread->state = DEAD;
82         return -1;
83     }
84
85     // thread is running
86     kernel_thread->state = RUNNING;
87     return 0;
88 }
```

پیاده سازی تابع `tkill()` :

این تابع برای متوقف کردن رشته‌ای که آیدی آن را گرفته است به کار می‌رود. ابتدا قفل ptable که رشته‌های مختلف در آن حضور دارند بدست می‌آوریم. سپس رشته بعدی که خواب است را بیدار می‌کنیم. از حالت SLEEPING به RUNNABLE تغییر می‌دهیم. اگر رشته اصلی برنامه بود یا موفق به اتمام رشته نشدیم 1- و در صورت موفقیت آمیز بودن 0 برگردانده می‌شود.

پیاده سازی تابع `int tsuspend(void)` :

که توسط خود رشته اجرا می‌شود. رشته در این حالت رشته متوقف و به خواب می‌رود. تا توسط رشته دیگر یا زمانبند بیدار شود.

پیاده سازی تابع `int tresume(int tid)` :

همانطور که گفته شد بیدار کردن یک رشته تنها توسط دیگر ممکن است.

پیاده سازی `init_sem()` با استفاده از semaphore برای به Deadlock نخوردن سر تصاحب منابع مشترک و همچنین انتظار به ترتیب برای جلوگیری از Starvation برای رشته‌ها، ضروری است.

پیاده سازی تابع `sched()` :

این تابع به صورت متناوب فراخوانی می‌شود که اگر لاک در دسترس بود و رشته خالی نبود، رشته در حال اجرا باشد. اگر نبود آماده اجرای رشته بعدی می‌شود.

چالش ها

اولین و واضح ترین چالش، پیاده سازی های حافظه و اختصاص آن ها به هر رشته، به طوری که توسط بقیه رشته ها به صورت همزمان دسترسی پیدا نکنند و همچنین اختصاص دادن استک به هر رشته بود. همچنین هنگام به هم رسیدن رشته ها حالت های پیش بینی نشده ای نیز وجود دارد که تقدم و تاخر گرفتن و رها کردن قفل نیازمند بررسی دقیق تر بود. همچنین مدیریت خطا برای زمان هایی که حافظه پر بود و یا حافظه رشته های قبلی آزاد نشده بود و حالات مشابه، به سختی قابل پیش بینی بود. بیدار کردن رشته توسط رشته ی دیگر و حالت بندی زمان های خواب هم نیازمند توجه هنگام پیاده سازی بود.

تست ها

تست اولیه clone در clone_join_test با پیاده سازی merge sort به صورت همزمان با دو رشته وجود دارد. تست حالت های اشتباهی که ممکن است پیش بیاید هم توسط یک تابع انجام می شود. در نهایت همروندی رشته ها و join آنها برای رعایت ترتیب هم تست می شود.

منابع

<https://github.com/kishanpatel22/xv6-kernel-threads/tree/master>

[/https://timothy.com/learning/xv6-book](https://timothy.com/learning/xv6-book)

<https://github.com/mit-pdos/xv6-public>